```
import pandas as pd
from google.colab import files


# load data
df = pd.read_csv("WhatnotDataRefined1023.csv")

print("Original dataset shape:", df.shape)

# Clean column names
df.columns = df.columns.astype(str).str.strip()
df = df.dropna(axis=1, how="all")

# Remove leftover unnamed columns
df = df.loc[
    :,
    ~df.columns.str.contains(r"^Unnamed", case=False)
]
#rows
df = df.dropna(how="all").copy()

# whitespace
for col in df.select_dtypes(include=["object", "string"]).columns:
    df[col] = df[col].str.strip()


df = df.replace(r"^\s*$", pd.NA, regex=True)

# numeric bids
for col in ["Starting Bid", "Ending Bid"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# style standardization
df["Style"] = df["Style"].str.upper()

# bid type standardization
df["Bid Type"] = df["Bid Type"].str.strip().str.lower()
df["Bid Type"] = df["Bid Type"].replace({
    "auction": "Auction",
    "sd": "SD"
})

# clothing type standardization
```

```
df["Type"] = df["Type"].str.strip().str.lower()

type_mapping = {
    "crew": "Crewneck",
    "crewneck": "Crewneck",
    "crew neck": "Crewneck",

    "quarterzip": "zipup",
    "quarter zip": "zipup",
    "quarter-zip": "zipup",

    "zipup hoodie": "hoodie",
    "zip up hoodie": "hoodie",
    "zip-up hoodie": "hoodie",

    "joggers": "pants",
    "shorts": "pants"
}

df["Type"] = df["Type"].replace(type_mapping)

# zipup standardization
quarterzip_mask = df["Type"].str.contains(
    r"quarter[\s-]*zip", case=False, na=False
)
df.loc[quarterzip_mask, "Type"] = "zipup"

# jacket standardization
is_varsity = df["Type"].str.contains(
    "varsity", case=False, na=False
)

contains_jacket = df["Type"].str.contains(
    "jacket", case=False, na=False
)

contains_detroit = df["Type"].str.contains(
    "detroit", case=False, na=False
)

jacket_mask = is_varsity | contains_jacket | contains_detroit
df.loc[jacket_mask, "Type"] = "Jacket"

# capitalize the first letter of every clothing type
df["Type"] = df["Type"].str.capitalize()
```

```
# wear and stain standardization
for col in ["Wear", "Stainage"]:
    df[col] = df[col].str.strip().str.lower()

# size needs to be text
df["Size"] = df["Size"].astype("string").str.strip()

# for review:
print("\nMissing values per column:")
print(df.isna().sum())

print("\nNegative bid records:")
print(df[
    (df["Starting Bid"] < 0) |
    (df["Ending Bid"] < 0)
])

print("\nEnding bids above $100:")
print(df[df["Ending Bid"] > 100])

# Check for exact duplicate rows
print("\nExact duplicate rows:", df.duplicated().sum())

# Display standardized clothing types and counts
print("\nStandardized clothing types:")
print(df["Type"].value_counts(dropna=False))

print(
    "\nTotal entries labeled Jacket:",
    (df["Type"] == "Jacket").sum()
)




print("\nColumn names:")
print(df.columns.tolist())


output_file = "/content/whatnot_data_cleaned23232.csv"
df.to_csv(output_file, index=False)

files.download(output_file)
```

```
Original dataset shape: (538, 13)

Missing values per column:
Style           0
Size            0
Starting Bid    0
Ending Bid      0
Type            0
Channel         0
Show Title      0
Bid Type        0
Wear            0
Stainage        0
dtype: int64

Negative bid records:
Empty DataFrame
Columns: [Style, Size, Starting Bid, Ending Bid, Type, Channel, Show Ti
Index: []

Ending bids above $100:
    Style Size  Starting Bid  Ending Bid      Type Channel  \
161    EM    M           5.0       112.0    Jacket   Mrtop
172    EM    M           5.0       180.0    Jacket   Mrtop
186     P    M           5.0       131.0    Jacket   Mrtop
205     P   XL           5.0       170.0  Crewneck   Mrtop
209     P    L           5.0       160.0    Jacket   Mrtop

              Show Title Bid Type      Wear Stainage
161        Carhartt Show       SD  moderate    clean
172        Carhartt Show       SD      none    clean
186  Premium Sports Show       SD      none    clean
205  Premium Sports Show       SD      none    clean
209  Premium Sports Show       SD      none   severe

Exact duplicate rows: 97

Standardized clothing types:
Type
Crewneck        157
Pants           133
Knit sweater     89
Hoodie           61
Jersey           27
Jacket           27
Windbreaker      13
Shirt            11
Zipup             9
Vest              3
Name: count, dtype: int64
```

```
Name: count, dtype: int64

Total entries labeled Jacket: 27

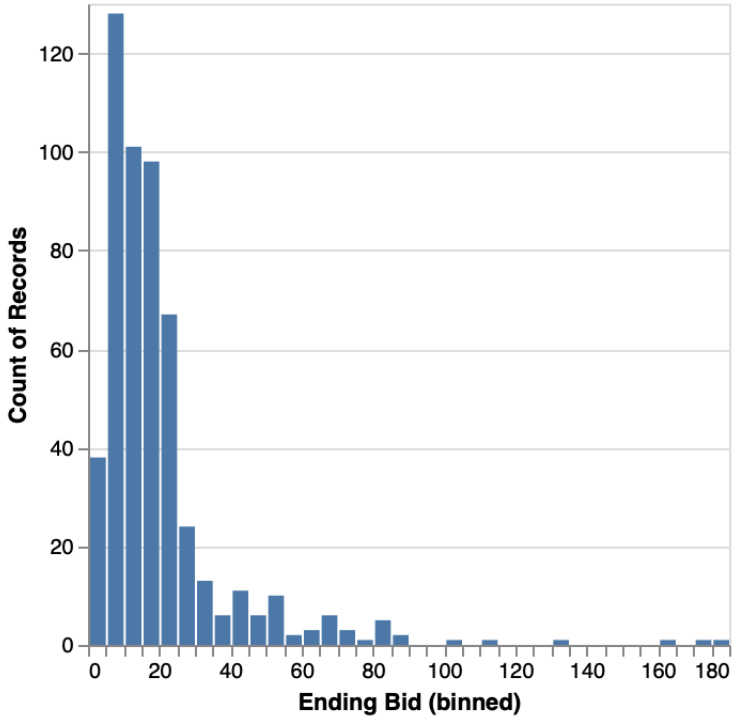
Column names:
['Style', 'Size', 'Starting Bid', 'Ending Bid', 'Type', 'Channel', 'Sho
```

```
import altair as alt
alt.data_transformers.disable_max_rows()

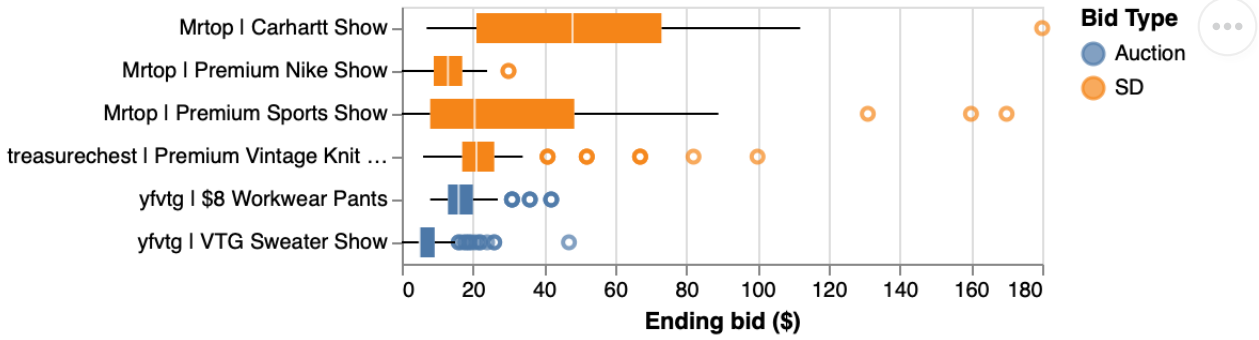
dv = df.dropna(subset=["Ending Bid"]).copy()                    # on
dv["stream"] = dv["Channel"] + " | " + dv["Show Title"]     # one live
LIGHT = alt.Scale(range=["#f0f4ff", "#7fa6e0"])                 # lig
```

```
print(dv["Ending Bid"].describe())
print("Items sold for $0:", (dv["Ending Bid"] == 0).sum())
alt.Chart(dv).mark_bar().encode(alt.X("Ending Bid:Q", bin=alt.Bin(maxl
```

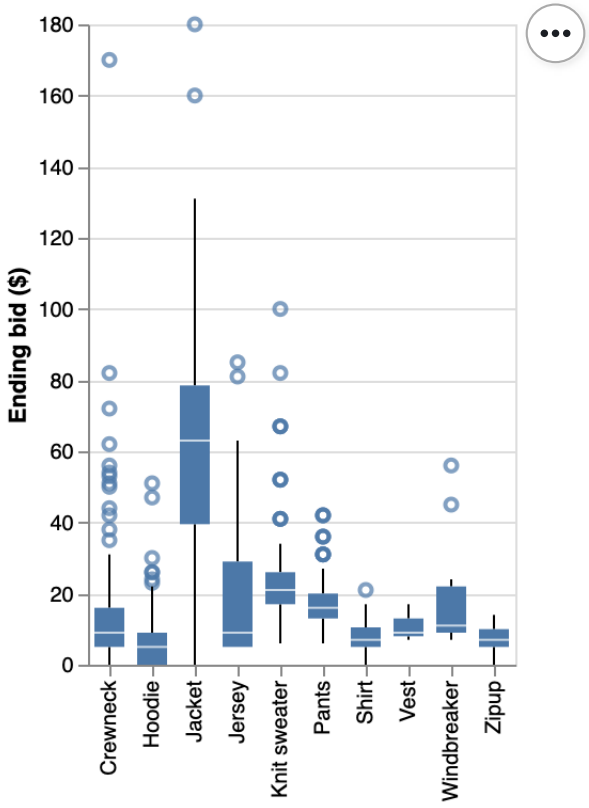
```
count    530.000000
mean      19.001887
std       20.737125
min        0.000000
25%        7.000000
50%       14.000000
75%       21.000000
max      180.000000
Name: Ending Bid, dtype: float64
Items sold for $0: 38
```



```
alt.Chart(dv).mark_boxplot().encode(
    x=alt.X("Ending Bid:Q", title="Ending bid ($)"),
    y=alt.Y("stream:N", title=None, sort=alt.EncodingSortField("Ending
    color="Bid Type:N", tooltip=["count()"])
```

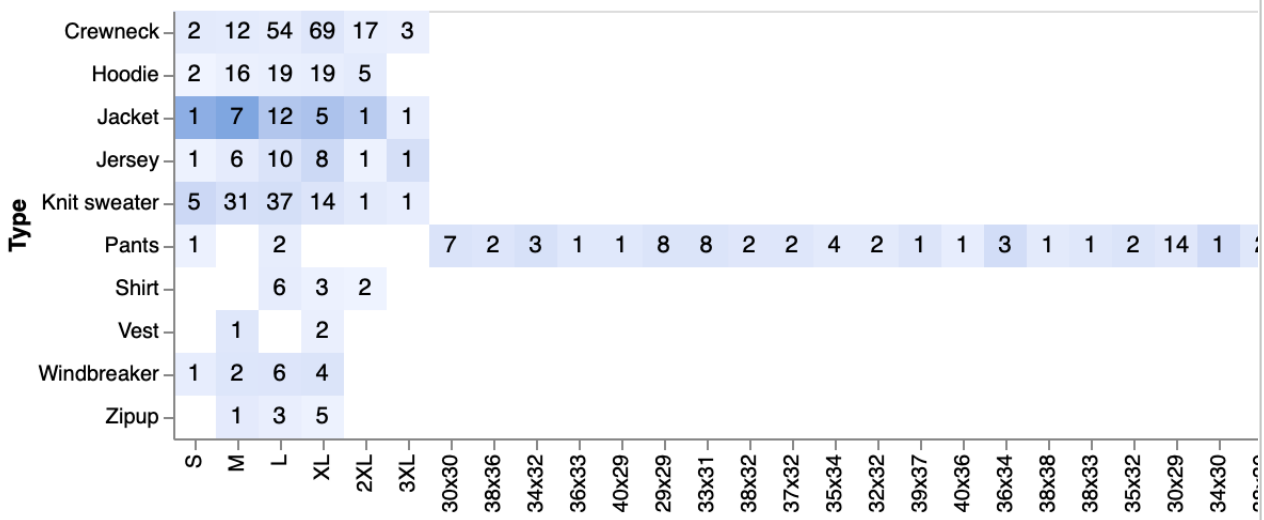



```
dropdown = alt.param(name="dropdown", value="Wear",
                bind=alt.binding_select(options=["Wear", "Stainage", '
alt.Chart(dv).add_params(dropdown).transform_calculate(group="datum[d
    x=alt.X("group:N", title=None), y=alt.Y("Ending Bid:Q", title="End
```

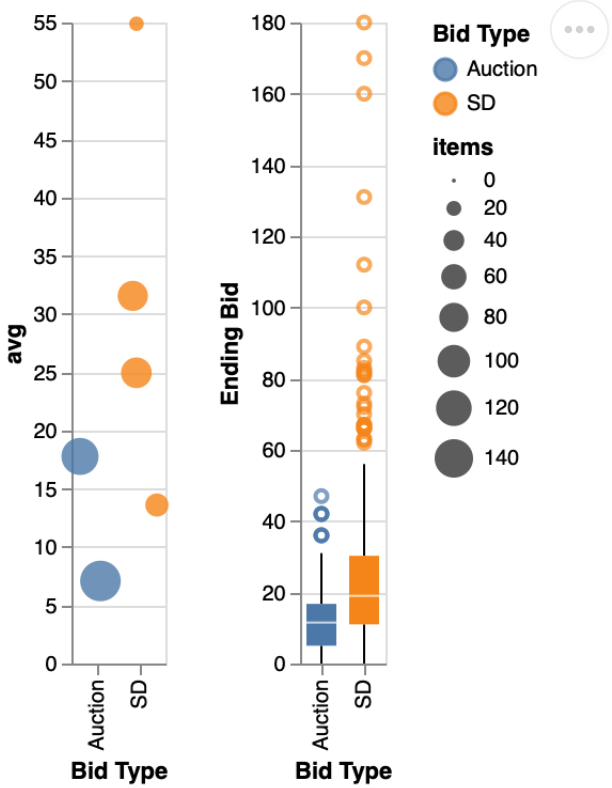


Compare by: Type

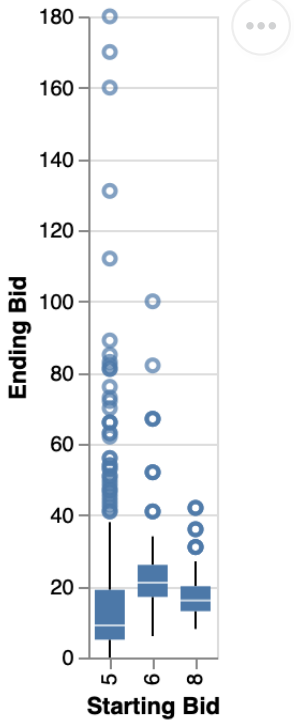
```
base = alt.Chart(dv).encode(x=alt.X("Size:O", sort=["XS", "S", "M", "I
base.mark_rect().encode(color=alt.Color("mean(Ending Bid):Q", title="I
base.mark_text().encode(text="count():Q")
```



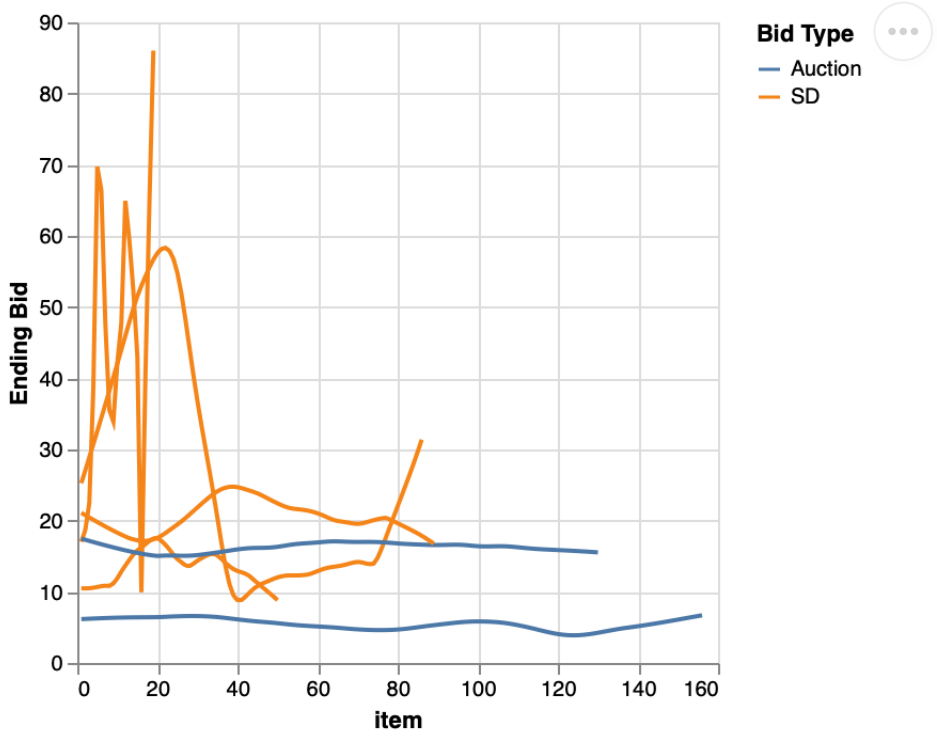
```
dots = (alt.Chart(dv).transform_aggregate(avg="mean(Ending Bid)", iter
        .transform_calculate(j="random()")
        .mark_circle(opacity=0.8).encode(x="Bid Type:N", xOffset="j:Q'
                                         tooltip=["stream:N", "avg:Q"
box = alt.Chart(dv).mark_boxplot().encode(x="Bid Type:N", y="Ending Bi
dots | box
```



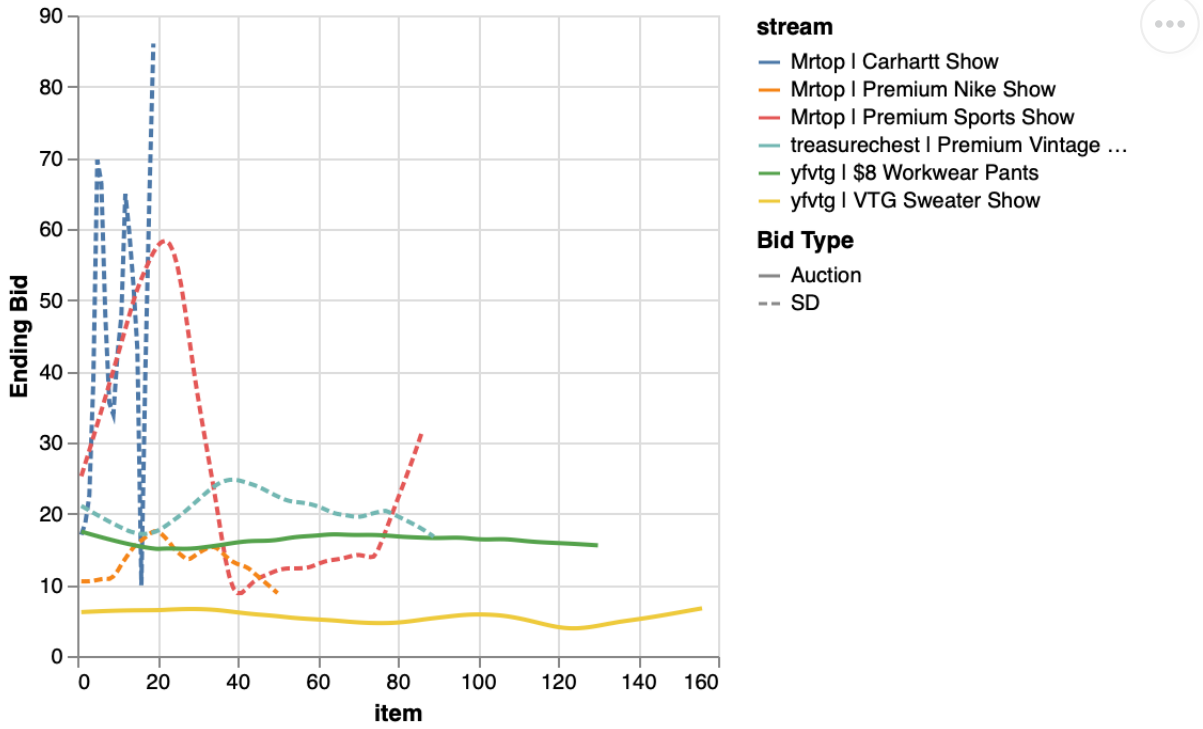
```
alt.Chart(dv).mark_boxplot().encode(x="Starting Bid:O", y="Ending Bid
```



```
(alt.Chart(dv)
   .transform_window(item="row_number()", groupby=["stream"])
   .transform_loess("item", "Ending Bid", groupby=["stream", "Bid Type
   .mark_line().encode(x="item:Q", y="Ending Bid:Q", color="Bid Type:N
```



```
(alt.Chart(dv)
   .transform_window(item="row_number()", groupby=["stream"])
   .transform_loess("item", "Ending Bid", groupby=["stream", "Bid Type
   .mark_line().encode(x="item:Q", y="Ending Bid:Q",
                       color="stream:N",
                       strokeDash="Bid Type:N",
                       detail="stream:N"))
```



```
(alt.Chart(dv)
   .transform_window(item="row_number()", groupby=["stream"])
   .transform_loess("item", "Ending Bid", groupby=["stream", "Bid Type"
   .mark_line().encode(x="item:Q", y="Ending Bid:Q", color="Bid Type:N"
   .properties(width=180, height=120)
   .facet("stream:N", columns=3))
```

